1. Smart Home Thermostat Controller Scenario: You are programming an automated smart thermostat. The system adjusts the room temperature based on the time of day ("morning", "afternoon", "evening", "night"), whether the house is occupied (True/False), and the current season ("summer", "winter"). Task: Write a program that takes these three inputs and determines the target temperature: ● If the house is unoccupied, set the temperature to $18^\circ\mathrm{C}$ in winter and $28^\circ\mathrm{C}$ in summer to save energy. ● If occupied during winter: set to $22^\circ\mathrm{C}$ in the morning/evening, $20^\circ\mathrm{C}$ in the afternoon, and $19^\circ\mathrm{C}$ at night. ● If occupied during summer: set to $23^\circ\mathrm{C}$ during the day (morning/afternoon/evening) and $21^\circ\mathrm{C}$ at night.

```
def get_target_temperature(time_of_day, occupied, season):
    if not occupied:
        return 18 if season == "winter" else 28

    if season == "winter":
        if time_of_day in ["morning", "evening"]:
            return 22
        elif time_of_day == "afternoon":
            return 20
        elif time_of_day == "night":
            return 19
    elif season == "summer":
        return 21 if time_of_day == "night" else 23
```

2. ATM Cash Dispenser with Note Availability Scenario: An ATM has a limited number of currency notes: 10 notes of $\$50$, 20 notes of $\$20$, and 30 notes of $\$10$. Task: Write a while loop simulation that prompts a user for a withdrawal amount. ● Validate that the amount is a multiple of 10 and does not exceed the total cash available in the ATM. ● Dispense the cash using the highest denominations possible first, updating the remaining note counts after each transaction. ● The loop should terminate when the ATM runs out of money or the user enters 0 to exit.

```
def simulate_atm():
    notes = {50: 10, 20: 20, 10: 30}

    while True:
        total_cash = sum(denom * count for denom, count in notes.items())
        if total_cash == 0:
            print("ATM out of money.")
            break

        req = int(input(f"Enter amount to withdraw (Multiple of 10, max ${total_cash}, 0 to exit): "))
        if req == 0:
            break
        if req % 10 != 0 or req > total_cash:
            print("Invalid amount.")
            continue

        dispensed = {50: 0, 20: 0, 10: 0}
        for denom in [50, 20, 10]:
            while req >= denom and notes[denom] > 0:
                req -= denom
                notes[denom] -= 1
                dispensed[denom] += 1

        print(f"Dispensed: {dispensed} | Remaining: {notes}")
```

3. Warehouse Inventory Reorder Alert System Scenario: A warehouse tracks its daily inventory changes over a 7-day period. If an item's stock drops below the reorder threshold (20 units), an urgent restock order of 50 units is triggered at the end of that day (arriving the next morning). However, if the stock hits 0 during the day, any excess demand is recorded as "lost sales." Task: Given an initial stock of 45 units and a list of daily customer demands demands = [12, 18, 25, 10, 30, 22, 15], write a for loop to simulate the week. Print the day number, starting stock, ending stock, whether a restock was triggered, and total lost sales at the end of the week.

```
stock = 45
demands = [12, 18, 25, 10, 30, 22, 15]
total_lost_sales = 0
pending_order = 0

for day, demand in enumerate(demands, 1):
    starting_stock = stock + pending_order
    pending_order = 0
```

```
    ending_stock = starting_stock - demand
    if ending_stock < 0:
        total_lost_sales += abs(ending_stock)
        ending_stock = 0

    restock_triggered = False
    if ending_stock < 20:
        pending_order = 50
        restock_triggered = True

    stock = ending_stock
    print(f"Day {day}: Start: {starting_stock}, End: {ending_stock}, Restock: {restock_triggered}")
```

```
Day 1: Start: 45, End: 33, Restock: False
Day 2: Start: 33, End: 15, Restock: True
Day 3: Start: 65, End: 40, Restock: False
Day 4: Start: 40, End: 30, Restock: False
Day 5: Start: 30, End: 0, Restock: True
Day 6: Start: 50, End: 28, Restock: False
Day 7: Start: 28, End: 13, Restock: True
Total Lost Sales: 0
```

4. Server Log Anomaly Detector Scenario: You are monitoring server response times (in milliseconds) streamed in a list: logs = [120, 135, 110, 520, 580, 610, 140, 125, 900, 115]. An anomaly alert is triggered only when three consecutive response times exceed $500\ \text{ms}$. Task: Write a loop to iterate through the log data. If three consecutive spikes are detected, print the exact index range where the anomaly occurred and halt further scanning using break. If the loop completes without finding three consecutive spikes, print "System Stable".

```
logs = [120, 135, 110, 520, 580, 610, 140, 125, 900, 115]
consecutive_spikes = 0
anomaly_found = False

for i in range(len(logs)):
    if logs[i] > 500:
        consecutive_spikes += 1
        if consecutive_spikes == 3:
            print(f"Anomaly detected at indices {i-2} to {i}")
            anomaly_found = True
            break
    else:
        consecutive_spikes = 0

if not anomaly_found:
    print("System Stable")
```

```
Anomaly detected at indices 3 to 5
```

5. Subscription Tier Upgrade Analyzer Scenario: A SaaS company tracks monthly active usage hours for a list of users. Users are on the "Basic" tier ($10\ \text{hours limit}$). Every hour over the limit costs $\$2.50$. If a user's overage charges exceed the cost of the "Pro" tier upgrade ($\$15$ difference) for two consecutive months, the system recommends an upgrade. Task: Given a dictionary where keys are usernames and values are lists of 6 monthly usage hours, use nested loops and if-else conditions to generate a list of users who should receive an upgrade recommendation.

```
usage_data = {
    "userA": [12, 18, 17, 10, 8, 9],
    "userB": [5, 6, 8, 9, 10, 5],
    "userC": [15, 14, 18, 20, 11, 17]
}
recommendations = []

for user, months in usage_data.items():
    consecutive_overage_months = 0
    for hours in months:
        overage_cost = max(0, hours - 10) * 2.50
        if overage_cost > 15:
            consecutive_overage_months += 1
            if consecutive_overage_months == 2:
                recommendations.append(user)
                break
        else:
            consecutive_overage_months = 0
```

```
19
```

```
Upgrade recommendations for: ['userA', 'userC']
```

6. E-Commerce Dynamic Pricing Engine Scenario: An online store calculates the final price of a cart based on multiple optional factors. Task: Write a function calculate_checkout(cart_total, tax_rate=0.08, discount_code=None, is_member=False): ● Apply a 10% discount if discount_code == "SAVE10", or a flat $\$20$ off if discount_code == "FLAT20" (only valid if cart_total >= 100). ● If is_member is True, apply an additional 5% discount on the discounted subtotal. ● Apply the tax_rate at the very end. Return the final rounded price.

```
def calculate_checkout(cart_total, tax_rate=0.08, discount_code=None, is_member=False):
    if discount_code == "SAVE10":
        cart_total *= 0.90
    elif discount_code == "FLAT20" and cart_total >= 100:
        cart_total -= 20

    if is_member:
        cart_total *= 0.95

    final_price = cart_total * (1 + tax_rate)
    return round(final_price, 2)
```

7. Automated Email Retry Mechanism Scenario: When sending automated reports, network glitches can cause failures. Task: Create a function connect_to_server(server_ip, port=8080, max_retries=3, timeout=5.0). Simulate a connection attempt inside the function (you can pass a dummy list of boolean statuses like [False, False, True]). The function should loop up to max_retries times. If it connects, return "Connected to

:". If all retries fail, return "Connection Timed Out after <max_retries * timeout> seconds".

```
def connect_to_server(server_ip, port=8080, max_retries=3, timeout=5.0):
    dummy_statuses = [False, False, True]

    for attempt in range(max_retries):
        status = dummy_statuses[attempt] if attempt < len(dummy_statuses) else False
        if status:
            return f"Connected to {server_ip}:{port}"

    return f"Connection Timed Out after {max_retries * timeout} seconds"
```

8. Loan EMI & Amortization Calculator Scenario: A fintech app needs a reusable function to calculate Equated Monthly Installments (EMI). Task: Write a function calculate_emi(principal, annual_rate=8.5, tenure_years=5, processing_fee_pct=1.0). ● Calculate the monthly interest rate $r = \frac{\text{annual_rate}}{12 \times 100}$ and total months $n = \text{tenure_years} \times 12$. ● Use the formula:

$$\text{EMI} = P \times r \times \frac{(1+r)^n}{(1+r)^n - 1}$$

● Return a dictionary containing monthly_emi, total_payable, and processing_fee.

```
def calculate_emi(principal, annual_rate=8.5, tenure_years=5, processing_fee_pct=1.0):
    r = (annual_rate / 12) / 100
    n = tenure_years * 12

    emi = principal * r * ((1 + r)**n) / (((1 + r)**n) - 1)
    processing_fee = principal * (processing_fee_pct / 100)
    total_payable = emi * n

    return {
        "monthly_emi": round(emi, 2),
        "total_payable": round(total_payable, 2),
        "processing_fee": round(processing_fee, 2)
    }
```

9. Text Sanitizer for NLP Pipeline Scenario: Before feeding customer reviews into a machine learning model, the text must be cleaned based on configurable rules. Task: Define a function sanitize_text(text, lowercase=True, remove_punctuation=True, stop_words=None). ● If stop_words is None, default it inside the function to ["the", "is", "at", "which", "on"]. ● Process the string according to the boolean flags, remove any words present in stop_words, and return the cleaned string.

```
import string

def sanitize_text(text, lowercase=True, remove_punctuation=True, stop_words=None):
    if stop_words is None:
        stop_words = ["the", "is", "at", "which", "on"]

    if lowercase:
        text = text.lower()

    if remove_punctuation:
        text = text.translate(str.maketrans('', '', string.punctuation))

    words = text.split()
    cleaned_words = [word for word in words if word not in stop_words]

    return " ".join(cleaned_words)
```

10. Multi-Branch Sales Aggregator (*args) Scenario: A retail chain has a varying number of pop-up stores reporting daily revenue. Some days 3 stores report; other days 10 stores report. Task: Write a function analyze_daily_sales(target_revenue, *branch_sales) that accepts a daily target followed by any number of branch revenue figures. The function should calculate the total revenue, average branch revenue, the number of branches that beat the average, and whether the overall target_revenue was met.

```
def analyze_daily_sales(target_revenue, *branch_sales):
    if not branch_sales:
        return None

    total_revenue = sum(branch_sales)
    avg_revenue = total_revenue / len(branch_sales)
    above_avg_count = sum(1 for sales in branch_sales if sales > avg_revenue)
    target_met = total_revenue >= target_revenue

    return {
        "total_revenue": total_revenue,
        "average_branch_revenue": round(avg_revenue, 2),
        "branches_beating_average": above_avg_count,
        "target_met": target_met
    }
```

11. Dynamic SQL Query Builder (**kwargs) Scenario: A backend developer needs a helper function to generate SELECT SQL queries dynamically based on user-applied filters on a dashboard. Task: Write a function build_query(table_name, **filters) that takes a table name and arbitrary keyword arguments representing column-value pairs (e.g., status="active", role="admin", min_age=25). ● Return a formatted SQL string: "SELECT * FROM users WHERE status='active' AND role='admin' AND min_age=25;".

● If no **filters are passed, return "SELECT * FROM ;".

```
def build_query(table_name, **filters):
    if not filters:
        return f"SELECT * FROM {table_name};"

    conditions = []
    for k, v in filters.items():
        if isinstance(v, str):
            conditions.append(f"{k}='{v}'")
        else:
            conditions.append(f"{k}={v}")

    where_clause = " AND ".join(conditions)
    return f"SELECT * FROM {table_name} WHERE {where_clause};"
```

12. Hospital Patient Triage Logger (*args & **kwargs) Scenario: An emergency room intake system records a patient's name, an arbitrary number of reported symptoms, and optional vital signs/metadata (like blood_pressure, heart_rate, allergies). Task: Write a function register_patient(patient_name, *symptoms, critical_flag=False, **vitals). ● Format and print a clean medical intake report. ● Automatically set critical_flag to True in the output if "chest pain" or "shortness of breath" is in *symptoms, or if heart_rate > 120 is passed in **vitals.

```
def register_patient(patient_name, *symptoms, critical_flag=False, **vitals):
    symptom_list = [s.lower() for s in symptoms]

    if "chest pain" in symptom_list or "shortness of breath" in symptom_list:
```

```
        critical_flag = True
    if vitals.get("heart_rate", 0) > 120:
        critical_flag = True

    print(f"--- Intake Report: {patient_name} ---")
    print(f"Critical Status: {'CRITICAL' if critical_flag else 'Stable'}")
    print(f"Symptoms: {', '.join(symptoms) if symptoms else 'None reported'}")
    print("Vitals:")
    for k, v in vitals.items():
```

13. Smart Data Pipeline Router (*args & **kwargs) Scenario: A data engineering pipeline takes multiple raw numerical datasets (lists) and applies configurable transformation rules passed as keyword arguments. Task: Write a function process_datasets(*datasets, **operations). ● Combine all lists passed in *datasets into a single flattened list. ● Check **operations for keys: if drop_negatives=True, remove numbers < 0; if scale_factor is provided, multiply all elements by that factor; if offset is provided, add that value to all elements. Return the processed list.

```
def process_datasets(*datasets, **operations):
    flattened = [item for sublist in datasets for item in sublist]

    if operations.get("drop_negatives"):
        flattened = [x for x in flattened if x >= 0]

    scale = operations.get("scale_factor")
    if scale is not None:
        flattened = [x * scale for x in flattened]

    offset = operations.get("offset")
    if offset is not None:
        flattened = [x + offset for x in flattened]

    return flattened
```

14. Restaurant Group Bill Splitter Scenario: Friends dining at a restaurant order shared appetizers (*shared_items prices) and individual mains (**individual_orders where key is the person's name and value is their main course price). Task: Write a function split_bill(tax_pct=5.0, tip_pct=10.0, *shared_items, **individual_orders). ● Divide the total cost of *shared_items equally among all people present in **individual_orders. ● Add each person's individual order cost, then apply tax_pct and tip_pct proportionally to each person's share. ● Return a dictionary mapping each person's name to their final payable amount.

```
def split_bill(tax_pct=5.0, tip_pct=10.0, *shared_items, **individual_orders):
    num_people = len(individual_orders)
    shared_per_person = sum(shared_items) / num_people if num_people > 0 else 0
    multiplier = 1 + (tax_pct / 100) + (tip_pct / 100)

    final_amounts = {}
    for person, main_price in individual_orders.items():
        subtotal = shared_per_person + main_price
        final_amounts[person] = round(subtotal * multiplier, 2)

    return final_amounts
```

15. Flight Search Sorting Engine Scenario: A travel aggregator fetches a list of flight dictionaries:

Python flights = [{"airline": "AirMax", "price": 350, "duration_hrs": 4.5, "stops": 1}, {"airline": "SkyJet", "price": 420, "duration_hrs": 2.5, "stops": 0}, {"airline": "CloudNine", "price": 350, "duration_hrs": 3.8, "stops": 1}, {"airline": "BudgetFly", "price": 290, "duration_hrs": 7.0, "stops": 2}]

Task: Use sorted() with lambda functions to generate three separate sorted lists:

1. Sorted by price ascending, and if prices are tied, by duration_hrs ascending.
2. Sorted by a custom "convenience score" calculated as $\text{price} + (\text{duration_hrs} \times 30) + (\text{stops} \times 50)$.

```
flights = [
    {"airline": "AirMax", "price": 350, "duration_hrs": 4.5, "stops": 1},
    {"airline": "SkyJet", "price": 420, "duration_hrs": 2.5, "stops": 0},
    {"airline": "CloudNine", "price": 350, "duration_hrs": 3.8, "stops": 1},
    {"airline": "BudgetFly", "price": 290, "duration_hrs": 7.0, "stops": 2}
]

# 1. Sorted by price ascending, then duration
```

```
sorted_by_price = sorted(flights, key=lambda x: (x["price"], x["duration_hrs"]))

# 2. Sorted by custom convenience score
sorted_by_score = sorted(flights, key=lambda x: x["price"] + (x["duration_hrs"] * 30) + (x["stops"] * 50))
```

16. Payroll Tax & Bonus Processor (map & filter with lambda) Scenario: HR has a list of employee tuples: employees = [("Alice", 85000, 4.8), ("Bob", 52000, 3.2), ("Charlie", 95000, 4.9), ("Diana", 60000, 3.9)], representing (name, salary, performance_rating). Task:
17. Use filter() and a lambda function to extract only employees eligible for a bonus (performance_rating >= 4.0).
18. Chain or pass that result into map() with a lambda function to grant them a 12% salary hike, returning a list of tuples (name, new_salary).

```
employees = [
    ("Alice", 85000, 4.8),
    ("Bob", 52000, 3.2),
    ("Charlie", 95000, 4.9),
    ("Diana", 60000, 3.9)
]

eligible = filter(lambda e: e[2] >= 4.0, employees)
promoted = list(map(lambda e: (e[0], round(e[1] * 1.12, 2)), eligible))
# Output: [('Alice', 95200.0), ('Charlie', 106400.0)]
```

17. IoT Sensor Calibration Factory (Closures & Lambdas) Scenario: Different temperature sensors in a factory have slight hardware drift and require custom linear calibration using the formula y = mx + c, where m is the scale factor and c is the offset. Task: Write a function create_calibrator(scale=1.0, offset=0.0) that returns a lambda function taking a raw sensor reading x and returning the calibrated reading. Demonstrate this by creating two calibrators (sensor_A and sensor_B) and applying them to a list of raw readings.

```
def create_calibrator(scale=1.0, offset=0.0):
    return lambda x: (scale * x) + offset

sensor_A = create_calibrator(scale=1.02, offset=-0.5)
sensor_B = create_calibrator(scale=0.98, offset=1.2)

raw_readings = [20.0, 22.5, 25.0]
calibrated_A = [sensor_A(r) for r in raw_readings]
calibrated_B = [sensor_B(r) for r in raw_readings]
```

18. Freelance Invoice Generator Scenario: A freelancer bills clients for fixed-price milestones, hourly tasks, and optional rush fees or discounts. Task: Write a function generate_invoice(client_name, hourly_rate=50, *milestone_payments, **hourly_tasks_and_modifiers): ● *milestone_payments contains flat amounts for completed phases.

● **hourly_tasks_and_modifiers contains task names mapped to hours worked (e.g., ui_design=12, backend_api=20), along with optional special keys like rush_delivery=True (adds 20% to total) and loyalty_discount=0.10 (subtracts 10%). ● Separate the task hours from the modifier flags using loops and if-else, use a lambda to compute hourly costs, and return a detailed invoice summary dictionary.

```
def generate_invoice(client_name, hourly_rate=50, *milestone_payments, **tasks_and_mods):
    milestone_total = sum(milestone_payments)

    rush = tasks_and_mods.pop("rush_delivery", False)
    discount = tasks_and_mods.pop("loyalty_discount", 0.0)

    hourly_costs = {task: hours * hourly_rate for task, hours in tasks_and_mods.items()}
    hours_total = sum(hourly_costs.values())

    subtotal = milestone_total + hours_total

    if rush:
        subtotal *= 1.20
    if discount:
        subtotal *= (1 - discount)

    return {
        "client": client_name,
        "milestones": milestone_total,
        "tasks": hourly_costs,
        "final_total": round(subtotal, 2)
    }
```

19. Automated Grading & Curve System Scenario: A university professor wants to grade students dynamically. Task: Write a function grade_students(passing_mark=50, curve_fn=None, *bonus_points, **student_scores): ● **student_scores holds student names and their raw exam scores (e.g., John=42, Emma=88). ● *bonus_points is a variable list of extra credit points awarded to the whole class (sum them up and add to every student's score). ● curve_fn is an optional lambda function (defaulting to lambda x: x if None is provided) applied to each student's score after bonus points (e.g., lambda x: min(100, x * 1.05)). ● Use a loop and if-elif-else to assign letter grades (A, B, C, F based on passing_mark) and return a dictionary of final results.

```
def grade_students(passing_mark=50, curve_fn=None, *bonus_points, **student_scores):
    if curve_fn is None:
        curve_fn = lambda x: x

    total_bonus = sum(bonus_points)
    results = {}

    for student, score in student_scores.items():
        adjusted_score = curve_fn(score + total_bonus)

        if adjusted_score >= 90: grade = "A"
        elif adjusted_score >= 80: grade = "B"
        elif adjusted_score >= passing_mark: grade = "C"
        else: grade = "F"

        results[student] = {"score": round(adjusted_score, 2), "grade": grade}

    return results
```

20. Rule-Based Fraud Detection Engine Scenario: A bank processes a batch of transactions and evaluates them against a dynamic set of fraud-detection rules (lambda functions). Task: Write a function evaluate_transactions(transactions, risk_threshold=2, *fraud_rules, **account_settings): ● transactions is a list of dictionaries (each having amount, country, hour). ● *fraud_rules is a variable number of lambda functions passed as arguments. Each lambda takes a transaction dict and account_settings dict, returning True if suspicious (e.g., lambda tx, acc: tx['amount'] > acc.get('max_limit', 5000)). ● Loop through each transaction and evaluate it against all *fraud_rules. If the number of triggered rules is $\geq \text{risk_threshold}$, flag the transaction as "BLOCKED", otherwise "APPROVED". Return the annotated transaction list.

```
def evaluate_transactions(transactions, risk_threshold=2, *fraud_rules, **account_settings):
    evaluated_txs = []

    for tx in transactions:
        triggered_rules = 0
        for rule in fraud_rules:
            if rule(tx, account_settings):
                triggered_rules += 1

        tx_copy = tx.copy()
        tx_copy["status"] = "BLOCKED" if triggered_rules >= risk_threshold else "APPROVED"
        tx_copy["rules_triggered"] = triggered_rules
        evaluated_txs.append(tx_copy)

    return evaluated_txs
```

Double-click (or enter) to edit

```
class Worker:
  company = "Acme"
  def __init__(self, id):
    self.id = id
w1 = Worker(1)
w2 = Worker(2)
Worker.company = "Global"
print(w1.company)
print(w2.company)

class Vault:
  def __init__(self):
    self._semi = "Protected"
    self.__priv = "Hidden"
v = Vault()
print(v._semi)
#print(v.__priv)
```

```
Global
Global
Protected
```

Double-click (or enter) to edit

```
# Class banayi
class Book:
    def __init__(self, title):
        # Jo title hum pass karenge, wo yahan object (self) me save ho jayega
        self.title = title

# Object banaya aur "WARM" title pass kiya
B1 = Book("WARM")

# Object ka title print karaya
print(B1.title)
```

```
WARM
```

```
class Product:
    def __init__(self, name, stock):
        self.name = name

        # 1. Private variable banana (Double underscore lagaya)
        self.__stock = stock

    # 2. Getter method likhna (Taki bahar se stock dekha ja sake)
    def get_stock(self):
        return f"{self.name} ka bacha hua stock: {self.__stock}"


# Object banaya
item = Product("Laptop", 50)

# SAHI TARIQA: Getter method ko call karke stock dekhna
print(item.get_stock())
# Output: Laptop ka bacha hua stock: 50

# GALAT TARIQA: Direct access prevent ho gaya hai (Isse code crash hoga)
# print(item.__stock)
# Ye AttributeError dega: 'Product' object has no attribute '__stock'
```

```
Laptop ka bacha hua stock: 50
```

```
class BankAccount:
    # 1. CLASS ATTRIBUTES (Global State)
    # These are shared by ALL instances of BankAccount.
    global_transaction_history = []
    total_bank_reserves = 0.0
    account_counter = 1000  # Used to generate unique account numbers

    def __init__(self, owner, initial_balance=0.0):
        # 2. INSTANCE ATTRIBUTES (Local State)
        # These belong ONLY to the specific object being created.
        self.owner = owner
        self.balance = initial_balance
```

```
13
14        # Generate a unique account number using the shared class attribute
15        BankAccount.account_counter += 1
16        self.account_number = BankAccount.account_counter
17
18        # Update global state
19        BankAccount.total_bank_reserves += initial_balance
20        self._log_transaction(f"NEW ACCOUNT: {self.owner} (Acc: {self.account_number}) opened with ${initial_balance:.2f}'
21
22    def deposit(self, amount):
23        if amount <= 0:
24            print("Deposit amount must be positive.")
25            return
26
27        # Update local state
28        self.balance += amount
29        # Update global state
30        BankAccount.total_bank_reserves += amount
31
32        self._log_transaction(f"DEPOSIT: {self.owner} deposited ${amount:.2f}")
33        print(f"Success! {self.owner}'s new balance: ${self.balance:.2f}")
34
35    def withdraw(self, amount):
36        if amount <= 0:
37            print("Withdrawal amount must be positive.")
38            return
39
40        if amount > self.balance:
41            self._log_transaction(f"FAILED WITHDRAWAL: {self.owner} attempted ${amount:.2f} (Insufficient funds)")
42            print("Error: Insufficient funds.")
43            return
44
45        # Update local state
46        self.balance -= amount
47        # Update global state
48        BankAccount.total_bank_reserves -= amount
49
50        self._log_transaction(f"WITHDRAWAL: {self.owner} withdrew ${amount:.2f}")
51        print(f"Success! {self.owner}'s new balance: ${self.balance:.2f}")
52
53    # 3. CLASS METHODS
54    # We use @classmethod to interact specifically with Class Attributes.
55    # 'cls' refers to the Class itself (BankAccount), just like 'self' refers to the object.
56    @classmethod
57    def _log_transaction(cls, message):
58        cls.global_transaction_history.append(message)
59
60    @classmethod
61    def print_global_report(cls):
62        print("\n=== GLOBAL BANK REPORT ===")
63        print(f"Total Bank Reserves: ${cls.total_bank_reserves:.2f}")
64        print(f"Total Active Accounts: {cls.account_counter - 1000}")
65        print("--- Transaction History ---")
66        for record in cls.global_transaction_history:
67            print(f" > {record}")
68        print("==========================\n")
69
70
71 # --- Testing the Code ---
72
73 # 1. Create two separate instances
74 acc1 = BankAccount("Alice", 500)
75 acc2 = BankAccount("Bob", 1000)
76
77 # 2. Perform localized actions
78 acc1.deposit(200)
79 acc2.withdraw(300)
80 acc1.withdraw(1000) # This will fail
81
82 # 3. Check the Global State
83 # Notice we call this on the Class itself, not the instance
84 BankAccount.print_global_report()
```

```
Success! Alice's new balance: $700.00
Success! Bob's new balance: $700.00
Error: Insufficient funds.

=== GLOBAL BANK REPORT ===
```

```
Total Bank Reserves: $1400.00
Total Active Accounts: 2
--- Transaction History ---
 > NEW ACCOUNT: Alice (Acc: 1001) opened with $500.00
 > NEW ACCOUNT: Bob (Acc: 1002) opened with $1000.00
 > DEPOSIT: Alice deposited $200.00
 > WITHDRAWAL: Bob withdrew $300.00
 > FAILED WITHDRAWAL: Alice attempted $1000.00 (Insufficient funds)
==========================
```

```python
class NounName:
    # ==========================================
    # ZONE 1: THE GLOBAL STATE (Class Attributes)
    # ==========================================
    # Rules:
    # 1. Variables written directly under the class.
    # 2. Use this ONLY for data that every single object must share.
    # 3. Access using the class name: NounName.variable

    shared_counter = 0
    company_name = "Global Inc."

    # ==========================================
    # ZONE 2: THE SETUP (Constructor)
    # ==========================================
    # Rules:
    # 1. ALWAYS named __init__(self, ...).
    # 2. 'self' must be the first parameter.
    # 3. This defines what data a single object needs the moment it is created.

    def __init__(self, public_data, secret_data):
        self.public_data = public_data  # Normal instance attribute

        # ZONE 3: PROTECTION (Encapsulation)
        # Add __ to hide data from the outside.
        self.__secret_data = secret_data

        # (Optional) Update Zone 1 global state when a new object is born
        NounName.shared_counter += 1

    # ==========================================
    # ZONE 3.5: THE GETTERS (@property)
    # ==========================================
    # Rules:
    # 1. Use if you made a variable private (__secret_data) in Zone 2.
    # 2. Allows outside code to read the secret safely without changing it.

    @property
    def secret_data(self):
        return self.__secret_data

    # ==========================================
    # ZONE 4: THE ACTIONS (Methods)
    # ==========================================
    # Rules:
    # 1. Regular actions take 'self' and modify local object data.
    # 2. @classmethods take 'cls' and modify Zone 1 global data.

    def regular_action(self, amount):
        # Modifies this specific object
        self.public_data += amount

    @classmethod
    def global_action(cls):
        # Modifies the entire blueprint
        print(f"Total objects created: {cls.shared_counter}")

# ==========================================
# USAGE (Outside the Class)
# ==========================================
# 1. Create the instance (Triggers Zone 2)
obj1 = NounName("Public Info", "Top Secret")

# 2. Trigger an action (Triggers Zone 4)
obj1.regular_action(5)

```

```
# 3. Read protected data (Triggers Zone 3.5)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipykernel_2176/2782554486.py in <cell line: 0>()
     63
     64 # 2. Trigger an action (Triggers Zone 4)
---> 65 obj1.regular_action(5)
     66
     67 # 3. Read protected data (Triggers Zone 3.5)

/tmp/ipykernel_2176/2782554486.py in regular_action(self, amount)
     49     def regular_action(self, amount):
     50         # Modifies this specific object
---> 51         self.public_data += amount
     52
     53     @classmethod

TypeError: can only concatenate str (not "int") to str
```

Next steps: Explain error

```
# 1. The Base Class (Parent)
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

    def honk(self):
        print(f"[{self.brand}] Beep beep! I am a standard vehicle.")

# 2. The Subclass (Child)
class Car(Vehicle):
    def __init__(self, brand, model):
        # super() calls the parent's __init__ so we don't have to rewrite self.brand = brand
        super().__init__(brand)
        self.model = model

    # 3. Method Overriding
    # By using the exact same method name as the parent, we replace its behavior
    def honk(self):
        print(f"[{self.brand} {self.model}] HONK HONK! Out of the way!")

# --- Testing the Code ---

# Create one of each
generic_cart = Vehicle("GenericBrand")
my_car = Car("Toyota", "Corolla")

# Call the exact same method on both
generic_cart.honk()
my_car.honk()
```

```
# 1. Create a Vehicle base
class Vehicle:
    def honk(self):
        print("Generic vehicle sound")

# 2. Derive a Car subclass
class Car(Vehicle):
    # 3. Override the honk method
    def honk(self):
        print("Beep beep! Car coming through!")

# Testing Q1
my_car = Car()
my_car.honk()
```

```
Beep beep! Car coming through!
```

```
class Airplane:
    def move(self):
        print("Flying through the sky!")

class Car:
    def move(self):
```

```
        print("Driving on the road!")

# Combine via multiple inheritance
class FlyingCar(Airplane, Car):
    pass

# Testing Q2
my_flying_car = FlyingCar()
my_flying_car.move() # Prints "Flying through the sky!" because Airplane is first in the parentheses

# Test Method Resolution Order (MRO)
print(FlyingCar.__mro__)
# Output shows it looks in FlyingCar -> Airplane -> Car -> object
```

```
Flying through the sky!
(<class '__main__.FlyingCar'>, <class '__main__.Airplane'>, <class '__main__.Car'>, <class 'object'>)
```

```
class ShoppingCart:
    def __init__(self, items=None):
        self.items = items if items is not None else []

    # Overload the + operator using the __add__ dunder method
    def __add__(self, other):
        # Create a new cart containing items from both carts
        combined_items = self.items + other.items
        return ShoppingCart(combined_items)

    def __str__(self):
        return f"Cart Contents: {self.items}"

# Testing Q3
cart1 = ShoppingCart(["Apples", "Bananas"])
cart2 = ShoppingCart(["Milk", "Bread"])

# Merge the two carts using the overloaded + operator
merged_cart = cart1 + cart2

print(merged_cart)
```

```
Cart Contents: ['Apples', 'Bananas', 'Milk', 'Bread']
```